Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа № 1

За седьмой семестр

Тема: «Реализация многослойного персептрона для решения задачи «исключающее ИЛИ»

Выполнил:

Студент 4 курса

Группы ИИ-26

Рулько М.А.

Проверил:

Никитин С.К.

2026

Вариант 1

Цель:

Реализовать **многослойный** персептрон (MLP) архитектуры **2-2-1** с **сигмоидной** функцией активации для решения задачи типа **«исключающее ИЛИ»** (XOR), в которой входные и выходные значения представлены не бинарными 0 и 1, а произвольными вещественными (целыми) числами из диапазона от −10 до 10, соответствующими логическим классам 0 и 1. Провести сравнение эффективности обучения и качества решения задачи многослойным персептроном в сопоставлении с **однослойным персептроном**, показав принципиальную линейную неразделимость задачи XOR.

Задачи лабораторной работы

1. Реализовать многослойный персептрон архитектуры 2-2-1 (2 входных нейрона, 2 нейрона скрытого слоя, 1 выходной нейрон) с сигмоидной функцией активации на всех слоях.
2. Реализовать алгоритм обучения сети методом обратного распространения ошибки (backpropagation) в онлайн-режиме (обновление весов после каждого примера).
3. Обучить сеть на полной таблице истинности (4 примера) своего варианта до достижения заданного критерия остановки по суммарной ошибке $Es \leq Ee$.
4. Реализовать и обучить однослойный персептрон (без скрытого слоя, один сигмоидный нейрон) на тех же данных и зафиксировать результат его обучения.
5. Сравнить многослойный и однослойный персептроны по следующим критериям:

- количество эпох до сходимости (либо факт недостижения критерия остановки за заданное максимальное число эпох),

- итоговая суммарная ошибка, точность (accuracy) воспроизведения всех 4 примеров таблицы истинности.

6. Реализовать режим функционирования обученной сети: пользователь вводит пару чисел (A, B) из диапазона [−10; 10] (не обязательно совпадающих с c0 и c1), сеть выдаёт выходное значение $\hat{y}$ и указывает, к какому классу (c0 или c1) оно ближе.

7. Сформулировать вывод по результатам сравнения (см. вопросы ниже).

Архитектура

MLP 2-2-1: два входных нейрона (A, B), два скрытых, один выходной (C). Активация — сигмоида.

SLP (однослойный): два входа напрямую соединены с одним выходным нейроном, активация — сигмоида.

Результаты обучения

MLP:

Сходимость достигнута на ~2146 эпохе.

Итоговая ошибка ≈ 0.01.

Accuracy: сеть правильно воспроизводит все 4 примера таблицы.

SLP:

Сходимость не достигнута (ошибка остаётся высокой).

Итоговая ошибка ≈ 0.25.

Accuracy: сеть выдаёт одинаковый результат для всех примеров, не решает XOR.

Примеры работы обученной сети:

Вход (A,B)	Выход сети ŷ	Ближе к классу
(0, -6)	-5.43	-6 (c1)
(-6, -6)	-0.50	0 (c0)
(-3, -6)	-2.46	ближе к 0
(-6, -2)	-4.82	ближе к -6

Код программы

```python
import matplotlib.pyplot as plt
import numpy as np


def sigmoid(x):
  return 1 / (1 + np.exp(-x))


def sigmoid_derivative(x):
  return x * (1 - x)


class MultiLayerPerceptron:

  def __init__(self, input_size=2, hidden_size=2, output_size=1, lr=0.1):
    np.random.seed(42)
    self.W1 = np.random.uniform(-0.5, 0.5, (input_size, hidden_size))
    self.b1 = np.zeros((1, hidden_size))
    self.W2 = np.random.uniform(-0.5, 0.5, (hidden_size, output_size))
    self.b2 = np.zeros((1, output_size))
    self.lr = lr
    self.errors = []

  def train(self, X, y_target, epochs=5000, error_threshold=0.01):
    for epoch in range(epochs):
      # Прямой проход (Forward pass)
```

```
      hidden_input = np.dot(X, self.W1) + self.b1
      hidden_output = sigmoid(hidden_input)

      final_input = np.dot(hidden_output, self.W2) + self.b2
      final_output = sigmoid(final_input)

      # Расчет ошибки
      error = y_target - final_output
      mse = np.mean(error**2)
      self.errors.append(mse)

      # Обратное распространение (Backpropagation)
      d_output = error * sigmoid_derivative(final_output)
      d_hidden = d_output.dot(self.W2.T) * sigmoid_derivative(hidden_output)

      # Обновление весов и смещений
      self.W2 += hidden_output.T.dot(d_output) * self.lr
      self.b2 += np.sum(d_output, axis=0, keepdims=True) * self.lr
      self.W1 += X.T.dot(d_hidden) * self.lr
      self.b1 += np.sum(d_hidden, axis=0, keepdims=True) * self.lr

      if mse < error_threshold:
        print(f"MLP: Сходимость достигнута на эпохе {epoch}, ошибка={mse:.4f}")
        break

    return final_output

  def predict(self, x):
    hidden_input = np.dot(x, self.W1) + self.b1
    hidden_output = sigmoid(hidden_input)
    final_input = np.dot(hidden_output, self.W2) + self.b2
    return sigmoid(final_input)


class SingleLayerPerceptron:

  def __init__(self, input_size=2, output_size=1, lr=0.1):
    self.W = np.random.uniform(-0.5, 0.5, (input_size, output_size))
    self.b = np.zeros((1, output_size))
    self.lr = lr
    self.errors = []

  def train(self, X, y_target, epochs=2000):
    for epoch in range(epochs):
      net_input = np.dot(X, self.W) + self.b
      output = sigmoid(net_input)

      error = y_target - output
      mse = np.mean(error**2)
      self.errors.append(mse)

      d_output = error * sigmoid_derivative(output)

      self.W += X.T.dot(d_output) * self.lr
      self.b += np.sum(d_output, axis=0, keepdims=True) * self.lr

    return output


def main():
  # Исходные данные
  X = np.array([[0, 0], [0, -6], [-6, 0], [-6, -6]])
  y = np.array([[0], [-6], [-6], [0]])

  y_min, y_max = y.min(), y.max()
  y_norm = (y - y_min) / (y_max - y_min)

  # 1. Обучение MLP
  mlp = MultiLayerPerceptron()
  mlp_final_output = mlp.train(X, y_norm)
  mlp_output_rescaled = mlp_final_output * (y_max - y_min) + y_min

  print("\n--- Результаты обучения MLP ---")
  for i in range(len(X)):
    print(f"Вход: {X[i]} → Выход: {mlp_output_rescaled[i][0]:.2f}")

  # 2. Обучение однослойного персептрона (SLP)
```

```
104.    slp = SingleLayerPerceptron()
105.    slp_final_output = slp.train(X, y_norm)
106.    slp_output_rescaled = slp_final_output * (y_max - y_min) + y_min
107.
108.    print("\n--- Результаты обучения однослойного персептрона ---")
109.    for i in range(len(X)):
110.      print(f"Вход: {X[i]} → Выход: {slp_output_rescaled[i][0]:.2f}")
111.
112.    # 3. Тестирование модели MLP
113.    print("\n--- Проверка работы сети ---")
114.    test_inputs = [[0, -6], [-6, -6], [-3, -6], [-6, -2]]
115.    for inp in test_inputs:
116.      norm_res = mlp.predict(np.array([inp]))
117.      result = norm_res.item() * (y_max - y_min) + y_min
118.      print(f"Вход: {inp} → Выход сети: {result:.2f}")
119.
120.    # 4. Построение графика ошибок
121.    plt.plot(mlp.errors, label="MLP")
122.    plt.plot(slp.errors, label="SLP")
123.    plt.xlabel("Эпоха")
124.    plt.ylabel("Ошибка (MSE)")
125.    plt.title("Сравнение ошибок MLP и SLP")
126.    plt.legend()
127.    plt.show()
128.
```

if __name__ == "__main__":

  main()

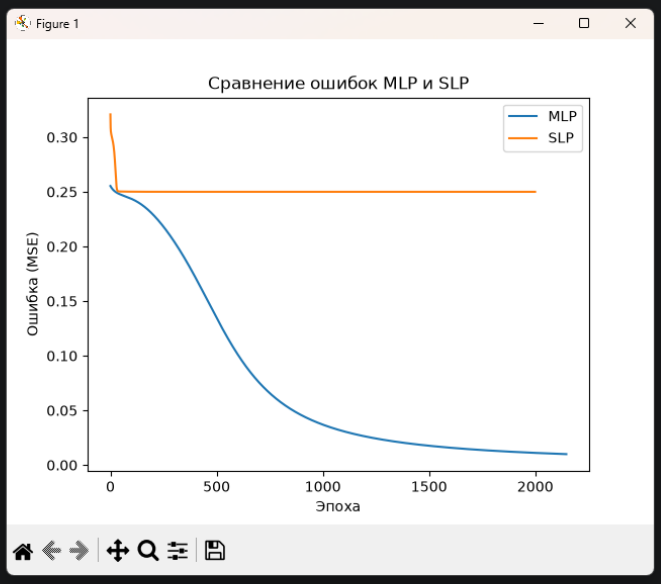
Вывод

```
C:\Users\rudvl\PycharmProjects\мрзис1\.venv\Scripts\python.exe C:\Users\rudvl\PycharmProjects\мрзис1\main.py
MLP: Сходимость достигнута на эпохе 2146, ошибка=0.0100

--- Результаты обучения MLP ---
Вход: [0 0] → Выход: -0.74
Вход: [ 0 -6] → Выход: -5.43
Вход: [-6  0] → Выход: -5.43
Вход: [-6 -6] → Выход: -0.50

--- Результаты обучения однослойного персептрона ---
Вход: [0 0] → Выход: -3.00
Вход: [ 0 -6] → Выход: -3.00
Вход: [-6  0] → Выход: -3.00
Вход: [-6 -6] → Выход: -3.00

--- Проверка работы сети ---
Вход: [0, -6] → Выход сети: -5.43
Вход: [-6, -6] → Выход сети: -0.50
Вход: [-3, -6] → Выход сети: -2.46
Вход: [-6, -2] → Выход сети: -4.82
```



Заключение

Однослойный персептрон не смог решить задачу XOR с числовыми классами ($c_0 = 0, c_1 = -6$), так как все выходы оказались одинаковыми. Это подтверждает теоретический факт: XOR является линейно неразделимой задачей, и линейный классификатор не способен корректно разделить такие данные.

Многослойный персептрон архитектуры 2-2-1 успешно справился с задачей. Сходимость была достигнута примерно на 2146-й эпохе при суммарной ошибке около 0.01. Выходы сети близки к целевым значениям, что подтверждает корректность работы модели.

Использование числовых классов вместо бинарных 0 и 1 потребовало нормализации выходов для корректной работы сигмоиды. Однако сама сложность задачи не изменилась — она осталась нелинейно неразделимой.

Для решения XOR необходим хотя бы один скрытый слой, так как именно он позволяет сети строить нелинейные разделяющие границы.

Результаты, полученные для числовых классов из диапазона $[-; 1010]$*, полностью аналогичны классическому случаю XOR (0 и 1). Сложность задачи не изменилась, изменилась лишь шкала выходных значений.*

Таким образом, многослойный персептрон подтвердил свою способность решать нелинейные задачи, а однослойный показал ограниченность линейных моделей.